

Asistente IA en Java

Documentación técnica completa

Versión	1.0
Estado	Planificación inicial
Interfaz	Consola
Lenguaje	Java

Documento base para el desarrollo incremental del proyecto.

1. Descripción general

Asistente IA en Java es una aplicación de consola desarrollada con Java cuyo objetivo es permitir que un usuario escriba una consulta, enviarla a un modelo de inteligencia artificial y mostrar la respuesta recibida.

El proyecto se plantea como un laboratorio práctico para aprender integración con servicios externos de IA utilizando Java puro, sin Spring Boot en la primera versión.

La aplicación comenzará con una implementación sencilla y evolucionará de manera incremental.

2. Objetivos del proyecto

Objetivo principal

Construir una aplicación Java capaz de comunicarse con una API de inteligencia artificial mediante HTTP.

Objetivos técnicos

- Practicar programación orientada a objetos.
- Consumir una API REST desde Java.
- Trabajar con solicitudes y respuestas HTTP.
- Procesar información en formato JSON.
- Utilizar variables de entorno para proteger credenciales.
- Separar la interfaz de usuario de la lógica de comunicación con la IA.
- Aplicar manejo básico de excepciones.
- Preparar el proyecto para una futura migración a Spring Boot.
- Incorporar Git durante el desarrollo.

3. Alcance inicial

La primera versión permitirá:

- Mostrar un menú por consola.
- Solicitar una pregunta al usuario.
- Enviar la pregunta a un servicio de IA.
- Recibir la respuesta del modelo.
- Mostrar la respuesta en consola.
- Permitir nuevas consultas hasta que el usuario decida salir.

Fuera del alcance inicial

En esta primera etapa no se incluirán:

- Interfaz gráfica.
- Aplicación web.
- Base de datos.
- Registro de usuarios.
- Autenticación.
- Historial persistente de conversaciones.
- Archivos adjuntos.

- Generación de imágenes.
- Uso de Spring Boot.
- Sistemas de agentes.
- RAG.
- Memoria permanente del asistente.

Estas funcionalidades podrán incorporarse en versiones posteriores.

4. Tecnologías

- Java.
- Java HttpClient.
- API REST de un proveedor de inteligencia artificial.
- JSON.
- Git.
- Variables de entorno.
- IDE por definir.

Versión de Java

El proyecto se desarrollará utilizando una versión moderna de Java compatible con HttpClient.

Versión sugerida:

Java 21 o superior

5. Arquitectura inicial

La aplicación se dividirá en componentes con responsabilidades separadas.

```
Usuario
|
v
AplicacionConsola
|
v
ServicioAsistente
|
v
ClienteIA
|
v
API de inteligencia artificial
```

Responsabilidades

AplicacionConsola

- Mostrar el menú.
- Leer las opciones del usuario.
- Solicitar la consulta.
- Mostrar la respuesta.
- Controlar el ciclo principal de ejecución.

ServicioAsistente

- Recibir la consulta del usuario.

- Validar que la consulta no esté vacía.
- Enviar la consulta al cliente de IA.
- Devolver la respuesta a la aplicación.

CienteIA

- Construir la solicitud HTTP.
- Agregar encabezados.
- Incluir la clave de acceso.
- Convertir la consulta a JSON.
- Enviar la solicitud.
- Procesar la respuesta.
- Informar errores de comunicación.

6. Estructura propuesta de paquetes

```

src
├── main
│   └── java
│       └── asistenteia
│           ├── aplicacion
│           │   └── Aplicacion.java
│           ├── cliente
│           │   ├── ClienteIA.java
│           │   └── ClienteIASimulado.java
│           ├── modelo
│           │   ├── Consulta.java
│           │   └── RespuestaIA.java
│           ├── servicio
│           │   └── ServicioAsistente.java
│           ├── excepcion
│           │   └── ComunicacionIAException.java
│           └── Main.java

```

Esta estructura es inicial y podrá modificarse según las necesidades del proyecto.

7. Modelo de clases inicial

Consulta

Representa la pregunta enviada por el usuario.

Atributos posibles:

texto

Responsabilidades:

- Almacenar el texto de la consulta.
- Evitar consultas nulas o vacías.

RespuestaIA

Representa la respuesta recibida desde el modelo.

Atributos posibles:

contenido
modelo
estado

En la primera versión podría utilizarse únicamente el contenido de la respuesta.

ClienteIA

Se propone utilizar una interfaz para desacoplar la aplicación del proveedor de inteligencia artificial.

```
public interface ClienteIA {  
  
    String enviarConsulta(String consulta);  
}
```

Esto permitirá utilizar diferentes implementaciones.

```
ClienteIASimulado  
ClienteOpenAI  
ClienteModeloLocal
```

ClienteIASimulado

Será utilizado durante la primera etapa para probar la arquitectura sin depender todavía de una API real. Podrá devolver respuestas predefinidas.

```
public class ClienteIASimulado implements ClienteIA {  
  
    @Override  
    public String enviarConsulta(String consulta) {  
        return "Respuesta simulada para: " + consulta;  
    }  
}
```

ServicioAsistente

Coordina la comunicación entre la aplicación y el cliente de IA.

```
public class ServicioAsistente {  
  
    private final ClienteIA clienteIA;  
  
    public ServicioAsistente(ClienteIA clienteIA) {  
        this.clienteIA = clienteIA;  
    }  
  
    public String procesarConsulta(String consulta) {  
        return clienteIA.enviarConsulta(consulta);  
    }  
}
```

8. Flujo principal

1. El usuario inicia la aplicación.
2. Se muestra el menú principal.
3. El usuario selecciona la opción para realizar una consulta.
4. La aplicación solicita el texto de la pregunta.
5. El servicio valida la consulta.
6. El cliente de IA genera o envía la solicitud.
7. Se obtiene una respuesta.
8. La respuesta se muestra en consola.
9. El usuario puede realizar otra consulta o salir.

9. Menú inicial

```
=====
ASISTENTE IA EN JAVA
=====
```

```
1 - Realizar una consulta
0 - Salir
```

Seleccione una opción:

Después de seleccionar la opción 1:

Escriba su consulta:

Respuesta esperada:

Respuesta del asistente:

[Contenido generado por la IA]

10. Integración HTTP

Cuando se incorpore una API real, el cliente utilizará HttpClient.

Componentes principales:

```
HttpClient
HttpRequest
HttpResponse
```

Flujo:

```
Consulta del usuario
    |
    v
Conversión a JSON
    |
    v
Solicitud HTTP POST
    |
    v
API de IA
    |
    v
Respuesta JSON
    |
    v
Extracción del contenido
    |
    v
Respuesta mostrada al usuario
```

11. Seguridad

La clave de acceso a la API no debe escribirse directamente en el código.

Ejemplo incorrecto:

```
String apiKey = "clave-secreta";
```

La clave deberá almacenarse en una variable de entorno.

```
AI_API_KEY
```

Lectura desde Java:

```
String apiKey = System.getenv("AI_API_KEY");
```

Si la variable no existe, la aplicación deberá informar el problema sin mostrar información sensible. La clave tampoco deberá incluirse en Git.

12. Manejo de errores

La aplicación deberá contemplar, como mínimo, los siguientes casos:

- Consulta vacía.
- Opción de menú inválida.
- Variable de entorno inexistente.
- Error de conexión.
- Tiempo de espera agotado.
- Respuesta HTTP inválida.
- JSON incorrecto o inesperado.
- Error interno del proveedor.
- Límite de uso de la API.
- Credenciales inválidas.

Los errores técnicos no deberían cerrar abruptamente la aplicación.

Ejemplo de mensaje para el usuario:

```
No fue posible comunicarse con el servicio de inteligencia artificial.  
Intente nuevamente más tarde.
```

El detalle técnico podrá registrarse por separado.

13. Validaciones iniciales

La consulta deberá:

- No ser nula.
- No estar vacía.
- No contener únicamente espacios.
- Respetar una longitud máxima definida.

Ejemplo de longitud inicial:

```
Máximo: 2000 caracteres
```

Esta restricción podrá modificarse según el proveedor utilizado.

14. Iteraciones propuestas

Iteración 1 - Estructura básica

- Crear el proyecto.
- Crear los paquetes.
- Crear el menú.
- Leer consultas desde consola.
- Crear la interfaz ClienteIA.
- Crear ClienteIASimulado.

- Mostrar una respuesta simulada.

Iteración 2 - Comunicación HTTP

- Incorporar HttpClient.
- Enviar una solicitud HTTP de prueba.
- Manejar códigos de estado.
- Controlar errores de conexión.

Iteración 3 - Integración con una API real

- Configurar la clave mediante variable de entorno.
- Construir el JSON requerido.
- Enviar consultas a un modelo.
- Procesar la respuesta JSON.
- Mostrar el contenido generado.

Iteración 4 - Mejoras de diseño

- Crear clases para consulta y respuesta.
- Incorporar excepciones propias.
- Separar responsabilidades.
- Mejorar validaciones.
- Agregar configuración externa.

Iteración 5 - Historial temporal

- Guardar las consultas durante la ejecución.
- Listar conversaciones anteriores.
- Limpiar el historial.
- Mantener el historial solo en memoria.

Iteración 6 - Migración futura

- Convertir la lógica en un servicio de Spring Boot.
- Exponer una API REST.
- Reemplazar la consola por una interfaz web.
- Incorporar persistencia.

15. Posibles mejoras futuras

- Guardar conversaciones en una base de datos.
- Crear distintos tipos de asistentes.
- Configurar instrucciones del sistema.
- Seleccionar diferentes modelos.
- Controlar la longitud de las respuestas.
- Incorporar respuestas en streaming.
- Analizar archivos.
- Crear un asistente especializado en Java.
- Integrar documentación propia mediante RAG.
- Agregar autenticación.

- Crear una interfaz web.
- Ejecutar modelos locales.
- Incorporar Docker.

16. Criterios de finalización de la primera versión

La primera versión se considerará terminada cuando:

- La aplicación pueda ejecutarse desde consola.
- El menú funcione correctamente.
- El usuario pueda escribir una consulta.
- La consulta sea validada.
- La aplicación pueda utilizar un cliente simulado.
- El diseño permita reemplazar el cliente simulado por uno real.
- Los errores básicos estén controlados.
- El código esté organizado en paquetes.
- El proyecto pueda almacenarse en un repositorio Git.

17. Estado actual

Estado: planificación inicial
Interfaz: consola
Lenguaje: Java
Persistencia: no incluida
Integración con IA real: pendiente
Git: pendiente de incorporación
Spring Boot: fuera del alcance inicial

18. Decisiones de diseño iniciales

- Se utilizará Java puro para comprender primero la comunicación HTTP.
- La interacción inicial será por consola.
- La aplicación dependerá de una interfaz ClientelA.
- Se comenzará con una implementación simulada.
- La integración real se agregará sin modificar la lógica principal.
- La clave de la API se administrará mediante variables de entorno.
- El proyecto se desarrollará por iteraciones pequeñas.
- No se incorporarán funcionalidades complejas hasta que la comunicación básica funcione.